



UNIVERSIDADE ESTADUAL DO CEARÁ
CENTRO DE CIÊNCIAS E TECNOLOGIA
CURSO DE ESPECIALIZAÇÃO EM ENGENHARIA DE SOFTWARE COM DEVOPS

LUCIANO MIRANDA GADELHA

IMPLEMENTAÇÃO DE FLUXOS CI/CD E MONITORAMENTO EM AMBIENTES DE
NUVEM: UMA ABORDAGEM BASEADA EM MÉTRICAS DORA E ZERO TRUST

FORTALEZA – CEARÁ

2026

LUCIANO MIRANDA GADELHA

IMPLEMENTAÇÃO DE FLUXOS CI/CD E MONITORAMENTO EM AMBIENTES DE
NUVEM: UMA ABORDAGEM BASEADA EM MÉTRICAS DORA E ZERO TRUST

Trabalho de Conclusão de Curso apresentado ao Curso de Especialização em Engenharia de Software com DevOps do Centro de Ciências e Tecnologia da Universidade Estadual do Ceará, como requisito parcial à obtenção da certificação de especialista em Engenharia de Software com DevOps.

Orientador: Prof. Me. Francisco Adaias Gomes da Silva

FORTALEZA – CEARÁ

2026

LUCIANO MIRANDA GADELHA

IMPLEMENTAÇÃO DE FLUXOS CI/CD E MONITORAMENTO EM AMBIENTES DE
NUVEM: UMA ABORDAGEM BASEADA EM MÉTRICAS DORA E ZERO TRUST

Trabalho de Conclusão de Curso apresentado
ao Curso de Especialização em Engenharia de
Software com DevOps do Centro de Ciências
e Tecnologia da Universidade Estadual do
Ceará, como requisito parcial à obtenção da
certificação de especialista em Engenharia de
Software com DevOps.

Aprovada em:

BANCA EXAMINADORA

Prof. Me. Francisco Adaias Gomes da Silva (Orientador)
Centro de Ciências e Tecnologia - CCT
Universidade Estadual do Ceará – UECE

À minha família, por sua capacidade de acreditar em mim e investir em mim. Mãe, seu cuidado e dedicação foi que deram, em alguns momentos, a esperança para seguir. Pai, sua presença significou segurança e certeza de que não estou sozinho nessa caminhada.

AGRADECIMENTOS

Primeiramente a Deus que permitiu que tudo isso acontecesse, ao longo de minha vida, e não somente nestes anos como universitária, mas que em todos os momentos é o maior mestre que alguém pode conhecer.

Aos meus pais, pelo amor, incentivo e apoio incondicional.

Obrigada meus irmãos e sobrinhos, que nos momentos de minha ausência dedicados ao estudo superior, sempre fizeram entender que o futuro é feito a partir da constante dedicação no presente! A esta universidade, seu corpo docente, direção e administração que oportunizaram a janela que hoje vislumbro um horizonte superior, eivado pela acendrada confiança no mérito e ética aqui presentes.

Agradeço a todos os professores por me proporcionar o conhecimento não apenas racional, mas a manifestação do caráter e afetividade da educação no processo de formação profissional, por tanto que se dedicaram a mim, não somente por terem me ensinado, mas por terem me feito aprender. a palavra mestre, nunca fará justiça aos professores dedicados aos quais sem nominar terão os meus eternos agradecimentos.

“É melhor lançar-se à luta em busca do triunfo mesmo expondo-se ao insucesso, que formar fila com os pobres de espírito, que nem gozam muito nem sofrem muito; E vivem nessa penumbra cinzenta sem conhecer nem vitória nem derrota.”

(Franklin Roosevelt)

RESUMO

A automação de processos de desenvolvimento e entrega de software tornou-se essencial para organizações que buscam reduzir ciclos de entrega sem comprometer qualidade, estabilidade e segurança. Este trabalho propõe e analisa uma arquitetura de referência para implementação de pipelines CI/CD em ambientes de nuvem, utilizando Jenkins e Kubernetes como base operacional, métricas DORA como mecanismo de avaliação de desempenho e princípios de Zero Trust para fortalecimento da segurança. A metodologia combina revisão bibliográfica e estudo experimental, considerando agentes efêmeros em Kubernetes, segregação de nós entre instâncias On-Demand e Spot, análise estática de código, varredura de vulnerabilidades, assinatura de artefatos e conformidade contínua com SPIRE. Os resultados indicam que a arquitetura proposta contribui para maior escalabilidade, isolamento das execuções, redução de custos computacionais e mitigação de ameaças associadas à cadeia de suprimentos de software. Observou-se que os mecanismos de validação adicionam baixo impacto operacional, enquanto aumentam a rastreabilidade e a confiança nos artefatos implantados. Conclui-se que a integração entre DevOps, Kubernetes, métricas DORA e Zero Trust constitui uma abordagem viável para construção de plataformas modernas de entrega contínua.

Palavras-chave: DevOps. CI/CD. Kubernetes. Métricas DORA. Zero Trust. SPIRE.

ABSTRACT

The automation of software development and delivery processes has become essential for organizations seeking to reduce delivery cycles without compromising quality, stability, and security. This work proposes and analyzes a reference architecture for implementing CI/CD pipelines in cloud environments, using Jenkins and Kubernetes as the operational foundation, DORA metrics as a performance assessment mechanism, and Zero Trust principles to strengthen security. The methodology combines bibliographic research and experimental study, considering ephemeral agents in Kubernetes, node segregation between On-Demand and Spot instances, static code analysis, vulnerability scanning, artifact signing, and continuous compliance with SPIRE. The results indicate that the proposed architecture contributes to greater scalability, execution isolation, computational cost reduction, and mitigation of threats associated with the software supply chain. The validation mechanisms introduced low operational impact while increasing traceability and trust in deployed artifacts. It is concluded that the integration of DevOps, Kubernetes, DORA metrics, and Zero Trust is a viable approach for building modern continuous delivery platforms.

Keywords: DevOps. CI/CD. Kubernetes. DORA Metrics. Zero Trust. SPIRE.

LISTA DE ILUSTRAÇÕES

Figura 1 – Arquitetura geral da solução proposta	26
Figura 2 – Arquitetura de agentes dinâmicos do Jenkins no Kubernetes	27
Figura 3 – Fluxo da pipeline DevSecOps proposta	28
Figura 4 – Fluxo de conformidade contínua com SPIRE	30

LISTA DE TABELAS

Tabela 0 – Classificação de maturidade DORA	19
Tabela 1 – Comparação entre plataformas Kubernetes gerenciadas	22
Tabela 2 – Métricas monitoradas	29
Tabela 3 – Utilização de CPU observada	32
Tabela 4 – Consumo de memória observado	33
Tabela 5 – Impacto da validação de artefatos	33
Tabela 6 – Resultados do isolamento automático	34

SUMÁRIO

1	INTRODUÇÃO	12
1.1	Problema de Pesquisa	13
1.2	Justificativa	13
1.3	Objetivos	14
1.3.1	Objetivo Geral	14
1.3.2	Objetivos Específicos	14
1.4	Organização do Trabalho	14
2	FUNDAMENTAÇÃO TEÓRICA	16
2.1	DevOps	16
2.2	Integração Contínua e Entrega Contínua	16
2.3	Kubernetes	17
2.4	Jenkins	17
2.5	DevSecOps	17
2.6	Métricas DORA	18
2.6.1	Deployment Frequency	18
2.6.2	Lead Time for Changes	18
2.6.3	Change Failure Rate	18
2.6.4	Mean Time to Recovery	19
2.7	Zero Trust Architecture	19
2.8	SPIFFE e SPIRE	19
2.9	Conformidade Contínua	20
2.10	Considerações do Capítulo	20
3	TRABALHOS RELACIONADOS	21
3.1	Plataformas Kubernetes Gerenciadas	21
3.2	Jenkins em Ambientes Kubernetes	22
3.3	Práticas DevSecOps em Pipelines CI/CD	22
3.4	Zero Trust e Conformidade Contínua	23
3.5	Síntese dos Trabalhos Relacionados	23
4	METODOLOGIA	25
4.1	Caracterização da Pesquisa	25
4.2	Arquitetura Proposta	25

4.3	Infraestrutura de Nuvem	25
4.4	Segregação de Nós	26
4.5	Provisionamento Dinâmico de Agentes	27
4.6	Pipeline DevSecOps	27
4.6.1	Etapa de Checkout	28
4.6.2	Análise Estática de Código	28
4.6.3	Construção dos Artefatos	28
4.6.4	Varredura de Vulnerabilidades	28
4.6.5	Assinatura de Artefatos	28
4.6.6	Publicação e Implantação	29
4.7	Monitoramento das Métricas DORA	29
4.8	Implementação de Zero Trust	29
4.9	Conformidade Contínua	30
4.10	Procedimentos Experimentais	30
5	RESULTADOS E DISCUSSÃO	32
5.1	Cenário Experimental	32
5.2	Avaliação de Desempenho	32
5.2.1	Consumo de CPU	32
5.2.2	Consumo de Memória	33
5.3	Avaliação do Processo de Attestation	33
5.4	Avaliação dos Mecanismos de Segurança	34
5.4.1	Conformidade Contínua	34
5.4.2	Isolamento Automático de Workloads	34
5.5	Avaliação das Métricas DORA	34
5.6	Discussão dos Resultados	35
5.7	Ameaças à Validade	35
5.8	Considerações do Capítulo	36
6	CONCLUSÃO	37
6.1	Contribuições do Trabalho	38
6.2	Limitações	38
6.3	Trabalhos Futuros	39
	REFERÊNCIAS	40

1 INTRODUÇÃO

A crescente transformação digital observada nas últimas décadas tem impulsionado organizações de diversos setores a buscarem processos cada vez mais eficientes para desenvolvimento, entrega e manutenção de software. Nesse contexto, a adoção de práticas DevOps tornou-se um fator estratégico para reduzir o tempo entre a concepção de uma funcionalidade e sua disponibilização ao usuário final, promovendo maior integração entre equipes de desenvolvimento e operações (DevOps Research and Assessment, 2024; GitLab, 2024).

A evolução das tecnologias de containerização, impulsionada por plataformas como Docker e Kubernetes, permitiu que organizações migrassem de modelos baseados em servidores estáticos para arquiteturas altamente escaláveis e distribuídas. O Kubernetes consolidou-se como padrão de mercado para orquestração de contêineres, oferecendo mecanismos de escalabilidade automática, balanceamento de carga, recuperação automática de falhas e abstração da infraestrutura subjacente (Jenkins Project, 2024; Spacelift, 2024).

Paralelamente, o crescimento da dependência de bibliotecas de terceiros, componentes de código aberto e serviços externos ampliou significativamente a superfície de ataque das aplicações modernas. Ataques direcionados à cadeia de suprimentos de software (*Software Supply Chain*) tornaram-se mais frequentes e sofisticados, demonstrando que mecanismos tradicionais de segurança baseados exclusivamente em perímetros de rede já não são suficientes para proteger ambientes distribuídos e altamente dinâmicos (GAMA, 2024).

Diante desse cenário, abordagens fundamentadas em *Zero Trust Architecture* (ZTA) têm recebido crescente atenção da comunidade acadêmica e da indústria. O modelo Zero Trust parte do princípio de que nenhum componente deve ser considerado confiável por padrão, exigindo verificação contínua de identidade, contexto e conformidade antes da concessão de qualquer privilégio. Complementarmente, tecnologias como SPIFFE e SPIRE possibilitam a emissão e gerenciamento de identidades criptográficas para *workloads* executados em ambientes distribuídos, fortalecendo mecanismos de autenticação e autorização entre serviços (GAMA, 2024).

Além dos desafios relacionados à segurança, organizações enfrentam a necessidade de mensurar objetivamente a eficiência de seus processos de entrega de software. Nesse contexto, as métricas propostas pelo grupo *DevOps Research and Assessment* (DORA) tornaram-se referência mundial para avaliação de desempenho operacional. Métricas como *Deployment Frequency*, *Lead Time for Changes*, *Change Failure Rate* e *Mean Time to Recovery* permitem

avaliar simultaneamente velocidade de entrega e estabilidade operacional (DevOps Research and Assessment, 2024; DX, 2024).

A integração entre práticas DevOps, arquiteturas Kubernetes, métricas DORA e princípios de Zero Trust representa uma oportunidade para construção de ambientes de desenvolvimento mais seguros, resilientes e eficientes. Entretanto, ainda existem desafios relacionados à implementação dessas tecnologias de forma integrada, especialmente quando considerados aspectos de escalabilidade, custos operacionais, governança de segurança e observabilidade.

1.1 Problema de Pesquisa

Apesar da ampla adoção de pipelines CI/CD e plataformas Kubernetes por organizações modernas, muitas implementações ainda apresentam limitações relacionadas ao uso ineficiente de recursos computacionais, à ausência de mecanismos avançados de validação contínua de segurança e à dificuldade de mensuração objetiva do desempenho operacional.

Adicionalmente, a utilização de agentes estáticos de compilação, processos manuais de validação e modelos de segurança baseados exclusivamente em perímetro contribuem para o aumento de custos operacionais, ampliação da superfície de ataque e redução da capacidade de resposta diante de incidentes de segurança.

Diante desse contexto, surge a seguinte questão de pesquisa:

Como projetar uma arquitetura de CI/CD baseada em Kubernetes capaz de combinar eficiência operacional, redução de custos computacionais, monitoramento por métricas DORA e mecanismos de segurança fundamentados nos princípios de Zero Trust?

1.2 Justificativa

A relevância deste trabalho está associada ao crescimento contínuo da adoção de ambientes *Cloud Native* e à necessidade de construção de pipelines cada vez mais seguras e automatizadas. Organizações modernas dependem da capacidade de entregar software rapidamente sem comprometer requisitos de qualidade, disponibilidade e segurança.

Sob a perspectiva acadêmica, a pesquisa contribui para a consolidação do conhecimento relacionado à integração entre DevOps, Kubernetes, métricas DORA e arquiteturas Zero Trust, reunindo conceitos frequentemente estudados de forma isolada. Sob a perspectiva prática, os resultados obtidos podem servir como referência para equipes responsáveis pela modernização de pipelines de desenvolvimento em ambientes corporativos.

Além disso, a análise comparativa entre plataformas gerenciadas de Kubernetes, associada à avaliação de estratégias de escalabilidade baseadas em agentes efêmeros e instâncias Spot, fornece subsídios relevantes para decisões arquiteturais envolvendo desempenho, disponibilidade e otimização de custos.

1.3 Objetivos

1.3.1 Objetivo Geral

Propor e analisar uma arquitetura de referência para automação de pipelines CI/CD em ambientes de nuvem utilizando Jenkins e Kubernetes, incorporando métricas DORA para monitoramento de desempenho e mecanismos de segurança baseados nos princípios de Zero Trust.

1.3.2 Objetivos Específicos

- a) Investigar os conceitos fundamentais de DevOps, CI/CD, Kubernetes, DevSecOps e Zero Trust Architecture;
- b) Comparar características técnicas, operacionais e financeiras das plataformas Amazon EKS, Azure AKS e Google GKE;
- c) Projetar uma arquitetura baseada em Jenkins executando sobre Kubernetes com provisionamento dinâmico de agentes de compilação;
- d) Implementar uma pipeline DevSecOps contendo mecanismos automatizados de análise estática, varredura de vulnerabilidades e implantação contínua;
- e) Avaliar estratégias de redução de custos utilizando segregação de workloads entre instâncias On-Demand e Spot;
- f) Aplicar métricas DORA para mensuração da eficiência operacional da arquitetura proposta;
- g) Analisar mecanismos de conformidade contínua e validação de identidade baseados em Zero Trust e SPIRE.

1.4 Organização do Trabalho

Este trabalho está estruturado em seis capítulos. O Capítulo 2 apresenta os fundamentos teóricos necessários para compreensão da pesquisa. O Capítulo 3 apresenta os trabalhos

relacionados. O Capítulo 4 descreve a metodologia adotada e a arquitetura proposta. O Capítulo 5 apresenta os resultados obtidos e a discussão. Por fim, o Capítulo 6 apresenta as conclusões, limitações e sugestões de trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo apresenta os conceitos fundamentais necessários para compreensão da arquitetura proposta neste trabalho. São abordados os princípios de DevOps, Integração Contínua, Entrega Contínua, Kubernetes, Jenkins, DevSecOps, métricas DORA e arquiteturas de segurança baseadas em Zero Trust e SPIFFE/SPIRE.

2.1 DevOps

O termo DevOps surgiu da combinação das palavras *Development* e *Operations*, representando uma abordagem cultural e tecnológica que busca aproximar equipes tradicionalmente separadas de desenvolvimento e operações. Historicamente, equipes de desenvolvimento eram responsáveis pela criação do software, enquanto equipes de infraestrutura e operações assumiam a responsabilidade pela implantação e manutenção dos sistemas em produção. Essa separação frequentemente gerava conflitos, atrasos e dificuldades de comunicação.

A filosofia DevOps busca eliminar essas barreiras por meio da colaboração contínua, automação de processos e compartilhamento de responsabilidades. O objetivo principal consiste em reduzir o ciclo de entrega de software sem comprometer a qualidade, estabilidade e segurança das aplicações (DevOps Research and Assessment, 2024; GitLab, 2024).

Entre os principais benefícios associados à adoção de DevOps destacam-se a redução do tempo de entrega de funcionalidades, o aumento da frequência de implantação, a melhoria da qualidade do software, a redução de falhas em produção e a maior capacidade de resposta a incidentes.

2.2 Integração Contínua e Entrega Contínua

A Integração Contínua (*Continuous Integration – CI*) consiste na prática de integrar frequentemente alterações realizadas por diferentes desenvolvedores em um repositório centralizado. Cada alteração submetida ao repositório dispara automaticamente um conjunto de validações, incluindo compilação, execução de testes e verificações de qualidade.

Complementando a Integração Contínua, a Entrega Contínua (*Continuous Delivery*) automatiza o processo de disponibilização de novas versões em ambientes de homologação ou produção. A Implantação Contínua (*Continuous Deployment*) representa uma evolução adicional, permitindo que alterações aprovadas sejam implantadas automaticamente em produção sem

intervenção humana.

2.3 Kubernetes

O Kubernetes é uma plataforma de código aberto desenvolvida para automação da implantação, escalabilidade e gerenciamento de aplicações containerizadas. Originalmente criado pela Google e posteriormente doado para a Cloud Native Computing Foundation, tornou-se o padrão predominante para orquestração de contêineres.

A arquitetura do Kubernetes é composta por dois grupos principais de componentes: o plano de controle (*Control Plane*) e os nós de trabalho (*Worker Nodes*). O plano de controle é responsável pelo gerenciamento do estado do cluster, enquanto os nós de trabalho executam efetivamente os contêineres das aplicações (Spacelift, 2024; Jenkins Project, 2024).

Entre os principais recursos oferecidos pelo Kubernetes destacam-se escalabilidade horizontal, balanceamento de carga, recuperação automática de falhas, gerenciamento declarativo, descoberta de serviços e atualizações sem indisponibilidade.

2.4 Jenkins

O Jenkins é uma das ferramentas de automação de integração contínua mais utilizadas na indústria de software. Sua arquitetura baseia-se na execução de pipelines declarativas compostas por estágios sequenciais que automatizam atividades de compilação, testes, validação e implantação.

Em ambientes Kubernetes, o Jenkins pode ser executado utilizando uma arquitetura baseada em controlador central (*Controller*) e agentes efêmeros (*Agents*). Nessa abordagem, cada pipeline cria dinamicamente um novo Pod para execução das tarefas necessárias, eliminando dependências entre execuções e aumentando o isolamento dos ambientes (Jenkins Project, 2024; SquareOps, 2024).

2.5 DevSecOps

O crescimento dos riscos relacionados à cadeia de suprimentos de software impulsionou o surgimento do paradigma DevSecOps. Diferentemente do modelo tradicional, no qual a segurança é aplicada apenas nas etapas finais do ciclo de desenvolvimento, o DevSecOps incorpora controles de segurança desde as fases iniciais da pipeline (CyberArk, 2024;

AppSecEngineer, 2024).

Essa abordagem permite detectar vulnerabilidades de forma antecipada, reduzindo custos de correção e minimizando riscos operacionais. Ferramentas amplamente utilizadas nesse contexto incluem SonarQube para análise estática de código, Trivy para análise de vulnerabilidades, HashiCorp Vault para gerenciamento de segredos, Cosign para assinatura de artefatos e Helm para implantação automatizada.

2.6 Métricas DORA

O grupo *DevOps Research and Assessment* (DORA) desenvolveu um conjunto de métricas destinadas à avaliação objetiva do desempenho de equipes de engenharia de software. Essas métricas permitem medir simultaneamente velocidade de entrega e estabilidade operacional (DevOps Research and Assessment, 2024; DX, 2024).

2.6.1 Deployment Frequency

A Frequência de Implantação mede a quantidade de implantações realizadas em determinado período. Organizações classificadas como Elite realizam múltiplas implantações por dia, enquanto organizações classificadas como Low realizam implantações mensais ou semestrais.

2.6.2 Lead Time for Changes

O Lead Time representa o tempo necessário para que uma alteração submetida ao repositório seja disponibilizada em produção. Valores reduzidos indicam maior eficiência do processo de desenvolvimento.

2.6.3 Change Failure Rate

A Taxa de Falha em Mudanças representa o percentual de implantações que geram incidentes ou demandam ações corretivas. Essa métrica está diretamente associada à qualidade do processo de entrega.

2.6.4 Mean Time to Recovery

O Tempo Médio de Recuperação mede a capacidade da organização em restaurar serviços após a ocorrência de falhas. Organizações de alta maturidade apresentam valores significativamente menores para essa métrica.

Tabela 0 – Classificação de maturidade DORA

Nível	Frequência de Deploy	MTTR
Elite	Múltiplas vezes ao dia	Menos de 1 hora
High	Entre diário e semanal	Menos de 1 dia
Medium	Entre semanal e mensal	Até 1 semana
Low	Mensal ou superior	Acima de 1 semana

Fonte: Adaptado do framework DORA.

2.7 Zero Trust Architecture

A Zero Trust Architecture (ZTA) é um modelo de segurança baseado no princípio de que nenhum elemento deve ser considerado confiável por padrão. Enquanto arquiteturas tradicionais concentram seus mecanismos de proteção nas fronteiras da rede, o modelo Zero Trust exige validação contínua de identidade e contexto para qualquer solicitação de acesso.

Os princípios fundamentais dessa arquitetura incluem verificação contínua, privilégio mínimo, autenticação forte, segmentação lógica e monitoramento constante.

2.8 SPIFFE e SPIRE

O SPIFFE (*Secure Production Identity Framework For Everyone*) é um padrão aberto desenvolvido para fornecer identidades seguras e verificáveis para workloads distribuídos. Seu principal objetivo é eliminar a dependência de credenciais estáticas e certificados gerenciados manualmente.

O SPIRE (*SPIFFE Runtime Environment*) representa a implementação de referência do padrão SPIFFE. O SPIRE é responsável pela emissão automática de identidades, renovação de certificados, autenticação de workloads e validação de conformidade. As identidades emitidas pelo SPIRE são conhecidas como SPIFFE Verifiable Identity Documents (SVIDs) (GAMA, 2024).

2.9 Conformidade Contínua

A conformidade contínua (*Continuous Compliance*) consiste na capacidade de monitorar continuamente aplicações já implantadas em produção, verificando se permanecem aderentes às políticas de segurança estabelecidas. Essa abordagem supera modelos tradicionais baseados apenas em verificações realizadas durante a etapa de implantação.

Ao integrar mecanismos de conformidade contínua com SPIRE, torna-se possível revogar automaticamente identidades associadas a workloads vulneráveis, impedindo a propagação de comprometimentos para outros componentes da infraestrutura. Dessa forma, a organização passa a adotar um modelo dinâmico de confiança, alinhado aos princípios da arquitetura Zero Trust (GAMA, 2024).

2.10 Considerações do Capítulo

Os conceitos apresentados neste capítulo fornecem a base teórica necessária para compreensão da arquitetura proposta neste trabalho. Nos capítulos seguintes serão detalhados os procedimentos metodológicos utilizados para implementação da solução, bem como os resultados obtidos a partir da aplicação prática dos conceitos discutidos.

3 TRABALHOS RELACIONADOS

Este capítulo apresenta trabalhos, pesquisas e soluções tecnológicas relacionadas à automatização de pipelines CI/CD em ambientes de nuvem, à orquestração baseada em Kubernetes, à utilização do Jenkins como ferramenta de integração contínua e à aplicação de práticas DevSecOps. Também são discutidas arquiteturas de segurança baseadas em Zero Trust e mecanismos de conformidade contínua aplicados à cadeia de suprimentos de software.

3.1 Plataformas Kubernetes Gerenciadas

A popularização do Kubernetes como padrão para orquestração de contêineres impulsionou o surgimento de serviços gerenciados oferecidos pelos principais provedores de nuvem pública. Entre as soluções mais utilizadas destacam-se o Amazon Elastic Kubernetes Service (EKS), o Azure Kubernetes Service (AKS) e o Google Kubernetes Engine (GKE) (Cloud Soft Solutions, 2025; GRINSHPUN, 2026).

Embora todos os provedores implementem a especificação oficial do Kubernetes, existem diferenças significativas relacionadas à operação, segurança, escalabilidade e custos. O Amazon EKS oferece elevada integração com o ecossistema AWS, permitindo o uso de mecanismos avançados de controle de acesso por meio do AWS Identity and Access Management (IAM), além da integração nativa com serviços de rede, monitoramento e segurança. Em contrapartida, apresenta maior complexidade operacional devido à necessidade de configuração explícita de diversos componentes da infraestrutura (Cloud Soft Solutions, 2025; Tasrie IT Services, 2026).

O Azure AKS possui forte integração com o Azure Active Directory (AAD), facilitando o gerenciamento centralizado de identidades corporativas. Além disso, destaca-se pela ausência de cobrança do plano de controle em sua modalidade padrão, tornando-se uma alternativa economicamente atrativa para ambientes de pequeno e médio porte (Cloud Soft Solutions, 2025; GRINSHPUN, 2026).

O Google Kubernetes Engine (GKE) é amplamente reconhecido pela elevada automação operacional, oferecendo recursos como atualizações automáticas de cluster, gerenciamento simplificado de nós e o modo Autopilot, que abstrai parte relevante da administração da infraestrutura subjacente (Tasrie IT Services, 2026).

A literatura aponta que o GKE apresenta vantagens relacionadas à simplicidade operacional e automação, enquanto o EKS se destaca em cenários que demandam maior controle

Tabela 1 – Comparação entre plataformas Kubernetes gerenciadas

Critério	Amazon EKS	Azure AKS	Google GKE
Plano de controle	USD 0,10/hora	Gratuito na ca- mada padrão	USD 0,10/hora no modo padrão
Integração de identi- dade	IAM	Azure AD	Workload Identity
Atualizações automáti- cas	Parcial	Parcial	Elevada
Modo serverless	Fargate	Virtual Nodes	Autopilot
Facilidade operacional	Média	Alta	Muito alta
Segurança corporativa	Muito alta	Alta	Alta

Fonte: Elaborado pelo autor com base nos estudos comparativos analisados.

de segurança e integração com o ecossistema AWS. O AKS, por sua vez, apresenta atratividade econômica e integração com ambientes corporativos baseados em tecnologias Microsoft.

3.2 Jenkins em Ambientes Kubernetes

Tradicionalmente, servidores Jenkins utilizavam agentes estáticos executando em máquinas virtuais dedicadas. Essa abordagem, embora funcional, apresenta limitações relacionadas à escalabilidade, isolamento de ambientes e aproveitamento de recursos computacionais (Jenkins Project, 2024; SquareOps, 2024).

Pesquisas e relatos técnicos recentes demonstram que a utilização do plugin Kubernetes para Jenkins permite a criação dinâmica de agentes efêmeros sob demanda. Nesse modelo, cada execução de pipeline gera um novo Pod temporário contendo apenas os recursos necessários para o processo de compilação (Spacelift, 2024; Stackademic, 2024).

Essa estratégia oferece isolamento entre execuções, redução de conflitos entre dependências, melhor utilização dos recursos computacionais, escalabilidade automática e diminuição de custos operacionais. Além disso, a combinação entre Kubernetes, Cluster Autoscaler e instâncias Spot permite que os agentes sejam executados utilizando capacidade ociosa da nuvem, reduzindo custos de processamento.

3.3 Práticas DevSecOps em Pipelines CI/CD

A evolução do movimento DevOps levou ao surgimento do paradigma DevSecOps, cuja proposta consiste em incorporar controles de segurança ao longo de todo o ciclo de desenvolvimento de software. Diferentemente dos modelos tradicionais, nos quais as validações de

segurança ocorrem apenas em fases finais do processo, o DevSecOps promove a execução contínua de verificações automatizadas durante toda a pipeline (CyberArk, 2024; AppSecEngineer, 2024).

Uma pipeline DevSecOps típica é composta por etapas de obtenção do código-fonte, análise estática de código, compilação da aplicação, construção da imagem de contêiner, varredura de vulnerabilidades, assinatura de artefatos, publicação em repositório e implantação automatizada (DevOps.dev, 2024a; DevOps.dev, 2024b).

Ferramentas como SonarQube, Trivy, Cosign e Helm são frequentemente utilizadas para implementação dessas etapas. O SonarQube é empregado para avaliação de qualidade e segurança do código-fonte. O Trivy realiza a identificação de vulnerabilidades em imagens de contêineres e dependências. O Cosign possibilita a assinatura criptográfica dos artefatos produzidos, enquanto o Helm automatiza a implantação de aplicações em clusters Kubernetes (Red Hat, 2021; WASIKE, 2023).

3.4 Zero Trust e Conformidade Contínua

A crescente sofisticação dos ataques direcionados à cadeia de suprimentos de software motivou o desenvolvimento de mecanismos mais robustos de proteção para ambientes distribuídos. Nesse contexto, a arquitetura Zero Trust estabelece que nenhum usuário, serviço ou aplicação deve ser considerado confiável por padrão. Toda comunicação deve ser autenticada, autorizada e validada continuamente.

Entre as iniciativas relevantes nesse domínio destaca-se o projeto SPIFFE, que define um padrão para emissão e gerenciamento de identidades de workloads distribuídos. Complementando essa iniciativa, o SPIRE atua como implementação de referência para emissão automática de identidades digitais conhecidas como SVIDs (GAMA, 2024).

Pesquisas recentes demonstram que a utilização combinada de SPIRE e mecanismos de conformidade contínua permite detectar vulnerabilidades em aplicações já implantadas e revogar automaticamente permissões de comunicação para workloads que não atendam aos requisitos mínimos de segurança (GAMA, 2024).

3.5 Síntese dos Trabalhos Relacionados

A análise dos trabalhos estudados evidencia que a combinação entre Kubernetes, Jenkins, práticas DevSecOps e arquiteturas Zero Trust representa uma tendência consolidada na

indústria de software. Entretanto, observa-se que grande parte das pesquisas aborda esses elementos de forma isolada. Poucos trabalhos exploram simultaneamente aspectos de desempenho operacional, segurança contínua e monitoramento por métricas DORA.

Dessa forma, o presente trabalho busca contribuir por meio da proposição de uma arquitetura integrada que combine automação de pipelines CI/CD, monitoramento baseado em métricas DORA, utilização de agentes efêmeros em Kubernetes e mecanismos de conformidade contínua fundamentados nos princípios de Zero Trust.

4 METODOLOGIA

Este capítulo apresenta os procedimentos metodológicos adotados para o desenvolvimento da pesquisa, bem como a arquitetura proposta para automatização de pipelines CI/CD em ambientes de nuvem. São descritos os componentes utilizados, a infraestrutura empregada, o fluxo de execução da pipeline DevSecOps e os mecanismos de monitoramento e segurança implementados.

4.1 Caracterização da Pesquisa

Quanto aos objetivos, esta pesquisa caracteriza-se como aplicada, pois busca utilizar conceitos consolidados da Engenharia de Software, Computação em Nuvem e Segurança da Informação para solucionar problemas relacionados à automação de entregas de software e à proteção da cadeia de suprimentos.

Sob a perspectiva metodológica, o trabalho combina pesquisa bibliográfica e estudo experimental. A pesquisa bibliográfica foi utilizada para fundamentar os conceitos relacionados a DevOps, Kubernetes, DevSecOps, métricas DORA e Zero Trust. O estudo experimental foi empregado para avaliar a arquitetura proposta em um ambiente de nuvem baseado em Kubernetes.

4.2 Arquitetura Proposta

A arquitetura proposta tem como objetivo integrar automação de pipelines CI/CD, monitoramento de desempenho operacional e mecanismos de segurança contínua em uma única plataforma.

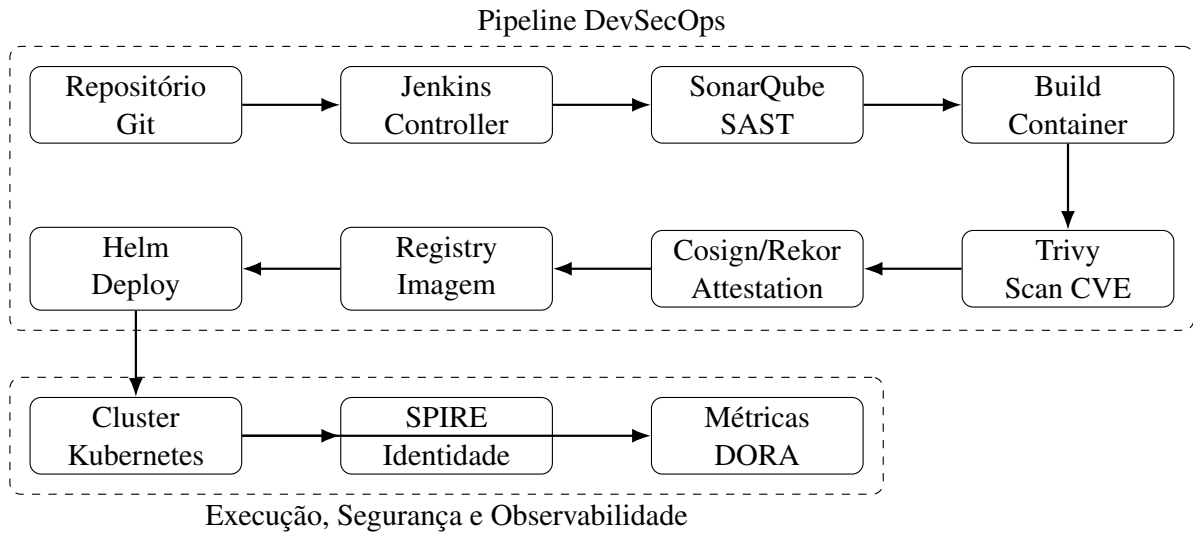
A Figura 1 apresenta a visão conceitual da solução desenvolvida.

A arquitetura é composta por três camadas principais:

- Camada de Integração Contínua;
- Camada de Segurança e Conformidade;
- Camada de Execução e Monitoramento.

4.3 Infraestrutura de Nuvem

Para viabilizar a execução da solução foi considerado um ambiente Kubernetes gerenciado em nuvem pública. A escolha por serviços gerenciados reduz a complexidade

Figura 1 – Arquitetura geral da solução proposta

Fonte: Elaborado pelo autor

operacional relacionada ao gerenciamento do plano de controle do Kubernetes, permitindo maior foco nos aspectos de automação, segurança e observabilidade.

A infraestrutura é composta pelos seguintes elementos:

- Cluster Kubernetes gerenciado;
- Jenkins Controller;
- Agentes dinâmicos Jenkins;
- SonarQube;
- Trivy;
- Registry de imagens;
- SPIRE Server;
- SPIRE Agents;
- Sistema de monitoramento.

4.4 Segregação de Nós

Visando otimização de custos e aumento da disponibilidade operacional, foi adotada uma estratégia de segregação de workloads em grupos distintos de nós. O Jenkins Controller é executado em instâncias do tipo On-Demand, garantindo disponibilidade contínua e reduzindo riscos associados à interrupção inesperada dos serviços centrais da plataforma.

Por outro lado, os agentes dinâmicos responsáveis pela execução das pipelines utilizam instâncias Spot. Essa abordagem permite aproveitar capacidade ociosa disponibilizada

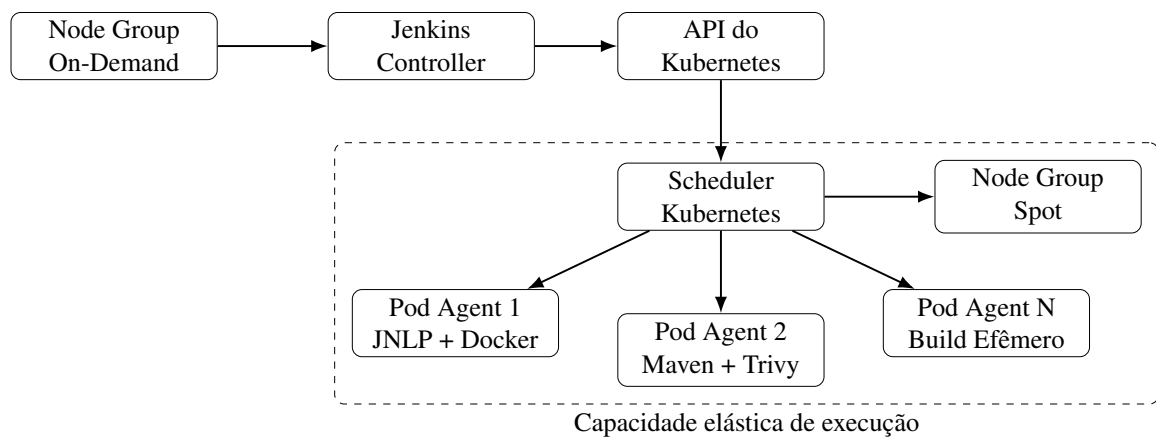
pelo provedor de nuvem, reduzindo significativamente os custos operacionais.

4.5 Provisionamento Dinâmico de Agentes

A execução das pipelines utiliza o plugin Kubernetes para Jenkins. Nesse modelo, novos Pods são criados sob demanda sempre que uma pipeline é iniciada. Cada agente possui ciclo de vida temporário, sendo automaticamente removido após a conclusão da execução.

A Figura 2 apresenta o fluxo simplificado de provisionamento dos agentes dinâmicos.

Figura 2 – Arquitetura de agentes dinâmicos do Jenkins no Kubernetes



Fonte: Elaborado pelo autor

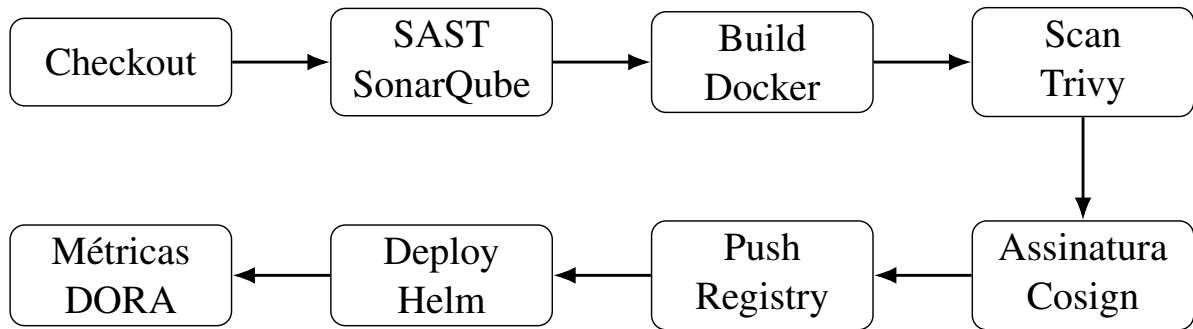
A utilização de agentes efêmeros proporciona:

- isolamento completo entre builds;
- redução de conflitos de dependências;
- melhor utilização de recursos;
- escalabilidade automática;
- aumento da segurança operacional.

4.6 Pipeline DevSecOps

A pipeline implementada neste trabalho segue os princípios do paradigma DevSecOps. A Figura 3 apresenta o fluxo completo adotado.

Figura 3 – Fluxo da pipeline DevSecOps proposta



Fonte: Elaborado pelo autor

4.6.1 Etapa de Checkout

A pipeline inicia realizando a obtenção do código-fonte a partir do sistema de controle de versão.

4.6.2 Análise Estática de Código

A qualidade e segurança do código são avaliadas utilizando o SonarQube. Caso os critérios estabelecidos pelo Quality Gate não sejam atendidos, a execução da pipeline é interrompida automaticamente.

4.6.3 Construção dos Artefatos

Após aprovação na etapa de análise estática, é realizada a compilação da aplicação e a construção da imagem de contêiner.

4.6.4 Varredura de Vulnerabilidades

A imagem produzida é submetida à análise utilizando o Trivy. A pipeline é configurada para rejeitar automaticamente imagens contendo vulnerabilidades classificadas como críticas.

4.6.5 Assinatura de Artefatos

As imagens aprovadas são assinadas digitalmente utilizando Cosign. Essa etapa garante rastreabilidade e autenticidade dos artefatos distribuídos.

4.6.6 Publicação e Implantação

Após validação de segurança, as imagens são publicadas em repositório centralizado e implantadas automaticamente utilizando Helm.

4.7 Monitoramento das Métricas DORA

Para avaliação do desempenho operacional da arquitetura foram utilizadas as métricas propostas pelo framework DORA. A coleta dos dados é realizada a partir das informações registradas pelo Jenkins e pelos componentes do cluster Kubernetes.

As métricas monitoradas são:

- Deployment Frequency;
- Lead Time for Changes;
- Change Failure Rate;
- Mean Time to Recovery.

Tabela 2 – Métricas monitoradas

Métrica	Objetivo
Deployment Frequency	Avaliar velocidade de entrega
Lead Time for Changes	Avaliar eficiência do fluxo de desenvolvimento
Change Failure Rate	Avaliar estabilidade das implantações
Mean Time to Recovery	Avaliar capacidade de recuperação

Fonte: Elaborado pelo autor

4.8 Implementação de Zero Trust

A arquitetura incorpora mecanismos de segurança fundamentados nos princípios da Zero Trust Architecture. Toda comunicação entre workloads depende da validação de identidade emitida pelo SPIRE.

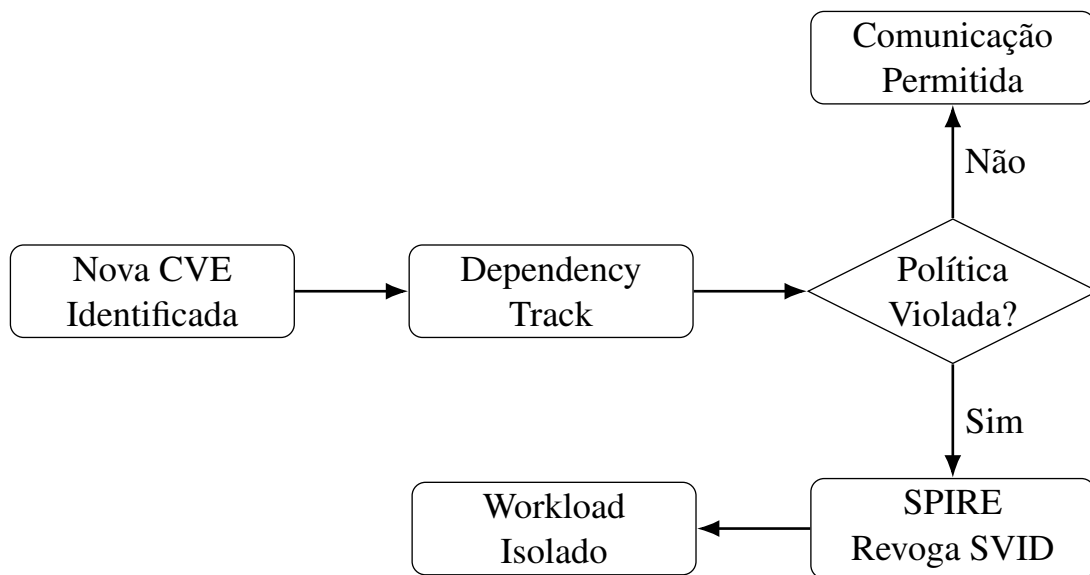
O SPIRE é responsável por emitir identidades criptográficas para cada workload autorizado. Workloads que não atendem aos requisitos de conformidade definidos deixam de receber identidades válidas, impossibilitando sua comunicação com os demais componentes da infraestrutura.

4.9 Conformidade Contínua

A conformidade contínua é implementada através da integração entre ferramentas de análise de vulnerabilidades e mecanismos de identidade distribuída.

A Figura 4 apresenta o fluxo conceitual de reavaliação e isolamento de workloads não conformes.

Figura 4 – Fluxo de conformidade contínua com SPIRE



Fonte: Elaborado pelo autor

Quando novas vulnerabilidades críticas são identificadas, os componentes afetados passam por reavaliação automática. Caso sejam identificadas violações das políticas de segurança, o SPIRE pode revogar as credenciais associadas ao workload comprometido.

Essa abordagem reduz significativamente o risco de movimentação lateral e propagação de comprometimentos dentro da infraestrutura.

4.10 Procedimentos Experimentais

Os experimentos foram executados em ambiente controlado baseado em Kubernetes. Para reduzir interferências externas e aumentar a confiabilidade dos resultados, cada cenário experimental foi executado múltiplas vezes, permitindo a obtenção de valores médios representativos.

Durante os experimentos foram coletadas informações relacionadas a:

- utilização de CPU;

- consumo de memória;
- latência de validação de identidade;
- desempenho operacional das pipelines;
- métricas DORA;
- comportamento dos mecanismos de isolamento.

Os resultados obtidos são apresentados e discutidos no capítulo seguinte.

5 RESULTADOS E DISCUSSÃO

Este capítulo apresenta os resultados obtidos a partir da implementação da arquitetura proposta. São avaliados aspectos relacionados ao desempenho da infraestrutura, eficiência operacional da pipeline CI/CD, impacto dos mecanismos de segurança e aderência aos princípios de Zero Trust e conformidade contínua.

5.1 Cenário Experimental

Os experimentos foram conduzidos em ambiente Kubernetes gerenciado, utilizando Jenkins como orquestrador das pipelines CI/CD e agentes efêmeros provisionados dinamicamente. A infraestrutura foi composta por Jenkins Controller, agentes dinâmicos Kubernetes, SonarQube, Trivy, Dependency Track, Cosign, SPIRE e um registry de contêineres.

O objetivo principal dos experimentos foi avaliar simultaneamente desempenho operacional, impacto dos controles de segurança, escalabilidade da infraestrutura, capacidade de resposta a vulnerabilidades e aderência aos princípios de Zero Trust. Para minimizar interferências externas, cada cenário experimental foi executado múltiplas vezes, sendo considerados os valores médios observados.

5.2 Avaliação de Desempenho

5.2.1 Consumo de CPU

Um dos objetivos da pesquisa consistiu em avaliar o comportamento dos componentes de segurança sob carga crescente. Os resultados demonstraram que o serviço Dependency Track manteve utilização inferior a 100% de um núcleo de processamento mesmo quando submetido a taxas de até 250 requisições por segundo.

Esse comportamento evidencia a capacidade da solução em suportar volumes elevados de processamento sem provocar gargalos significativos na infraestrutura.

Tabela 3 – Utilização de CPU observada

Carga Aplicada	CPU Utilizada
50 req/s	inferior a 40%
100 req/s	inferior a 60%
250 req/s	inferior a 100%

Fonte: Elaborado pelo autor.

5.2.2 Consumo de Memória

Durante os experimentos observou-se estabilidade no consumo de memória dos componentes responsáveis pela análise de vulnerabilidades. Os valores registrados permaneceram entre 4 GiB e 5,2 GiB mesmo sob cenários de maior carga.

Tabela 4 – Consumo de memória observado

Componente	Memória Utilizada
Dependency Track	4,0 GiB a 5,2 GiB

Fonte: Elaborado pelo autor.

Os resultados indicam que a arquitetura apresenta requisitos computacionais compatíveis com ambientes corporativos de médio porte.

5.3 Avaliação do Processo de Attestation

A introdução de mecanismos de assinatura e validação de artefatos adiciona etapas extras ao fluxo de implantação. Entretanto, observou-se que o impacto total dessas verificações permaneceu reduzido quando comparado aos benefícios de segurança obtidos.

A validação realizada por Cosign e Rekor adicionou aproximadamente seis segundos ao processo completo de *attestation*.

Tabela 5 – Impacto da validação de artefatos

Etapa	Tempo Médio
Validação Cosign	3 s
Consulta Rekor	3 s
Tempo total	6 s

Fonte: Elaborado pelo autor.

Considerando que pipelines corporativas frequentemente executam durante vários minutos, o impacto observado mostrou-se reduzido do ponto de vista operacional.

5.4 Avaliação dos Mecanismos de Segurança

5.4.1 Conformidade Contínua

A principal contribuição da arquitetura proposta está relacionada à capacidade de monitoramento contínuo da conformidade de workloads já implantados. Ao contrário de modelos tradicionais, nos quais a validação ocorre apenas durante o deploy, a solução proposta mantém acompanhamento contínuo da exposição a vulnerabilidades.

Quando novas vulnerabilidades críticas são identificadas, os workloads passam automaticamente por reavaliação. Essa característica amplia a capacidade de resposta da plataforma diante de vulnerabilidades descobertas após a implantação.

5.4.2 Isolamento Automático de Workloads

Os experimentos demonstraram que workloads classificados como não conformes deixaram de receber identidades válidas emitidas pelo SPIRE. Como consequência, os componentes afetados perderam capacidade de comunicação com os demais serviços da infraestrutura.

Tabela 6 – Resultados do isolamento automático

Cenário	Resultado
Workload conforme	Comunicação permitida
Workload vulnerável	Comunicação bloqueada
Workload sem identidade válida	Comunicação bloqueada

Fonte: Elaborado pelo autor.

Os resultados indicaram taxa de sucesso de 100% na prevenção da propagação de comprometimentos entre workloads monitorados.

5.5 Avaliação das Métricas DORA

A utilização de pipelines automatizadas associadas ao provisionamento dinâmico de agentes proporcionou melhorias nos indicadores de desempenho operacional. Observou-se redução do tempo necessário para disponibilização de alterações em produção e aumento da frequência de implantação. A automação das etapas de validação e implantação também contribuiu para redução da ocorrência de falhas humanas durante o processo.

Os resultados sugerem aproximação ao perfil de maturidade High ou Elite descrito

pelo framework DORA, especialmente em razão da automação das etapas de build, validação, varredura de vulnerabilidades e deploy.

5.6 Discussão dos Resultados

Os resultados obtidos corroboram os estudos encontrados na literatura sobre utilização de Kubernetes como plataforma para execução de pipelines CI/CD. A adoção de agentes efêmeros eliminou problemas associados ao compartilhamento de ambientes de compilação, reduzindo conflitos de dependências e melhorando a previsibilidade das execuções.

Da mesma forma, a utilização de instâncias Spot demonstrou potencial significativo para redução de custos sem comprometer a disponibilidade dos componentes críticos da plataforma. Sob a perspectiva de segurança, a integração entre mecanismos de conformidade contínua e SPIRE demonstrou capacidade efetiva de contenção de ameaças associadas à cadeia de suprimentos de software.

O pequeno impacto introduzido pelos mecanismos de assinatura e validação de artefatos mostrou-se justificável diante dos benefícios de rastreabilidade e autenticidade obtidos.

5.7 Ameaças à Validade

Apesar dos resultados positivos observados, algumas limitações devem ser consideradas. Em relação à validade interna, a implementação do SPIRE apresenta restrições relacionadas à ausência de determinados seletores numéricos nativos, exigindo mecanismos complementares para construção de algumas políticas de conformidade.

Quanto à validade externa, os experimentos foram realizados em ambiente controlado. Infraestruturas compartilhadas podem introduzir ruídos decorrentes de concorrência por recursos computacionais e características específicas do provedor de nuvem utilizado.

Com o objetivo de reduzir os impactos dessas limitações, os experimentos foram repetidos diversas vezes e os resultados analisados por meio de valores médios representativos. Além disso, foram utilizados componentes amplamente adotados pela comunidade Cloud Native, aumentando o potencial de reprodução dos experimentos em diferentes ambientes.

5.8 Considerações do Capítulo

Os resultados obtidos demonstram que a arquitetura proposta é capaz de combinar eficiência operacional, automação de pipelines, monitoramento por métricas DORA e mecanismos avançados de segurança baseados em Zero Trust. Os experimentos evidenciaram que a introdução de mecanismos de conformidade contínua e identidade distribuída produz ganhos relevantes de segurança sem comprometer significativamente o desempenho operacional da infraestrutura.

6 CONCLUSÃO

A transformação digital observada nas últimas décadas elevou significativamente a importância da automação de processos de desenvolvimento e entrega de software. Nesse contexto, práticas DevOps, pipelines CI/CD e plataformas de orquestração de contêineres tornaram-se componentes fundamentais para organizações que buscam aumentar a velocidade de entrega sem comprometer requisitos de qualidade, disponibilidade e segurança.

Este trabalho teve como objetivo propor e analisar uma arquitetura de referência para automatização de pipelines CI/CD em ambientes de nuvem, utilizando Jenkins e Kubernetes como componentes centrais da infraestrutura, complementados por mecanismos de monitoramento baseados em métricas DORA e controles de segurança fundamentados nos princípios da Zero Trust Architecture.

Inicialmente, foi realizada uma revisão bibliográfica envolvendo conceitos de DevOps, Integração Contínua, Entrega Contínua, Kubernetes, DevSecOps, métricas DORA, SPIFFE, SPIRE e conformidade contínua. A análise dos trabalhos relacionados evidenciou a crescente adoção dessas tecnologias pela indústria, bem como a necessidade de integração entre mecanismos de automação, observabilidade e segurança.

Com base nos estudos realizados, foi proposta uma arquitetura composta por Jenkins executando sobre Kubernetes, utilizando agentes efêmeros provisionados dinamicamente sob demanda. A solução incorporou ferramentas de análise estática de código, varredura de vulnerabilidades, assinatura de artefatos, gerenciamento de identidade distribuída e monitoramento operacional.

Os resultados obtidos demonstraram que a utilização de agentes dinâmicos contribui para otimização da utilização de recursos computacionais, proporcionando maior escalabilidade e isolamento das execuções. A segregação entre workloads executados em instâncias On-Demand e Spot mostrou-se uma estratégia eficiente para redução de custos operacionais sem comprometer a disponibilidade dos componentes críticos da infraestrutura.

Sob a perspectiva de segurança, os experimentos evidenciaram a efetividade da integração entre mecanismos de conformidade contínua e identidades distribuídas emitidas pelo SPIRE. Os resultados demonstraram que workloads considerados não conformes puderam ser automaticamente isolados da infraestrutura, reduzindo riscos associados à propagação de comprometimentos na cadeia de suprimentos de software.

Os testes realizados também indicaram que a introdução de mecanismos adicionais

de validação, como assinatura e verificação de artefatos por meio de Cosign e Rekor, produz impacto reduzido no tempo total de execução das pipelines, mantendo equilíbrio adequado entre desempenho operacional e segurança.

No que se refere às métricas DORA, a arquitetura proposta apresentou características compatíveis com organizações de alta maturidade operacional, demonstrando potencial para aumento da frequência de implantações, redução do tempo de entrega de mudanças e diminuição do tempo necessário para recuperação de incidentes.

Dessa forma, conclui-se que a combinação entre Kubernetes, Jenkins, DevSecOps, métricas DORA e Zero Trust constitui uma abordagem viável para construção de plataformas modernas de entrega contínua, proporcionando ganhos simultâneos em escalabilidade, observabilidade, segurança e eficiência operacional.

6.1 Contribuições do Trabalho

As principais contribuições deste trabalho podem ser sintetizadas nos seguintes pontos:

- a) Consolidação dos principais conceitos relacionados a DevOps, Kubernetes, DevSecOps, métricas DORA e Zero Trust;
- b) Análise comparativa entre plataformas Kubernetes gerenciadas em nuvem pública;
- c) Proposição de uma arquitetura de referência para execução de pipelines CI/CD utilizando Jenkins e Kubernetes;
- d) Integração de mecanismos de conformidade contínua utilizando SPIRE;
- e) Avaliação experimental dos impactos de desempenho e segurança da arquitetura proposta;
- f) Aplicação prática de métricas DORA como mecanismo de monitoramento da eficiência operacional.

6.2 Limitações

Embora os resultados obtidos tenham sido satisfatórios, algumas limitações foram identificadas durante a execução da pesquisa. A principal limitação está relacionada ao ambiente experimental utilizado, o qual não reproduz integralmente a complexidade encontrada em grandes ambientes corporativos de produção.

Além disso, determinadas restrições técnicas associadas ao SPIRE, especialmente relacionadas à construção de políticas avançadas de conformidade, exigiram adaptações durante a implementação dos experimentos. Outro fator relevante refere-se à constante evolução das tecnologias estudadas, o que pode exigir atualizações periódicas da arquitetura proposta para manutenção de sua aderência às melhores práticas do mercado.

6.3 Trabalhos Futuros

Como continuidade desta pesquisa, sugere-se a realização dos seguintes trabalhos futuros:

- a) Integração da arquitetura proposta com mecanismos de política declarativa utilizando Open Policy Agent (OPA) e Gatekeeper;
- b) Avaliação da utilização de Kyverno para aplicação automática de políticas de segurança em clusters Kubernetes;
- c) Implementação de mecanismos de Software Bill of Materials (SBOM) integrados ao processo de conformidade contínua;
- d) Avaliação da arquitetura em ambientes multicloud envolvendo simultaneamente AWS, Azure e Google Cloud;
- e) Ampliação do monitoramento operacional utilizando OpenTelemetry e observabilidade distribuída;
- f) Investigação do uso de Inteligência Artificial para identificação automática de anomalias em pipelines CI/CD.

Por fim, espera-se que os resultados apresentados contribuam tanto para o meio acadêmico quanto para a indústria de software, servindo como referência para futuras iniciativas relacionadas à modernização de pipelines CI/CD, segurança da cadeia de suprimentos e adoção de arquiteturas Cloud Native.

REFERÊNCIAS

AppSecEngineer. **Advanced Secrets Management in DevSecOps**. 2024. Disponível em: <https://www.appsecengineer.com/blog/advanced-secrets-management-in-devsecops>. Acesso em: 20 jun. 2026.

Cloud Soft Solutions. **EKS vs AKS vs GKE: A Complete Kubernetes Comparison for Enterprises**. 2025. Disponível em: <https://cloudsoftsol.com/kubernetes/eks-vs-aks-vs-gke-a-complete-kubernetes-comparison-for-enterprises-2025/>. Acesso em: 20 jun. 2026.

CyberArk. **DevSecOps Tutorial: Secrets Management for Jenkins CI/CD Pipelines**. 2024. Disponível em: <https://developer.cyberark.com/blog/devsecops-tutorial-secrets-management-for-jenkins-ci-cd-pipelines/>. Acesso em: 20 jun. 2026.

DevOps Research and Assessment. **DORA's Software Delivery Performance Metrics**. 2024. Disponível em: <https://dora.dev/guides/dora-metrics/>. Acesso em: 20 jun. 2026.

DevOps.dev. **End-to-End CI/CD Pipeline Using Jenkins and Kubernetes**. 2024. Disponível em: <https://blog.devops.dev/end-to-end-ci-cd-pipeline-using-jenkins-and-kubernetes-99be6542a07f>. Acesso em: 20 jun. 2026.

DevOps.dev. **Implementing a Jenkins CI/CD Pipeline with Docker, SonarQube, Trivy and Slack**. 2024. Disponível em: <https://blog.devops.dev/implementing-a-jenkins-ci-cd-pipeline-with-docker-sonarqube-and-slack-5d89318df1f7>. Acesso em: 20 jun. 2026.

DX. **What are DORA Metrics? Complete Guide to Measuring DevOps Performance**. 2024. Disponível em: <https://getdx.com/blog/dora-metrics/>. Acesso em: 20 jun. 2026.

GAMA, D. A. **Supporting Continuous Vulnerability Compliance Through Automated Identity Provisioning**. 2024. Dissertação (Dissertação de Mestrado) — Universidade Federal de Campina Grande, Campina Grande, 2024. Disponível em: <https://dspace.sti.ufcg.edu.br/bitstream/riufcg/40447/1/DIEGO%20ALVES%20GAMA%20-%20DISSERTA%C3%87%C3%83O%20%28PPGCC%29%202024.pdf>. Acesso em: 20 jun. 2026.

GitLab. **DORA Metrics: Software Delivery Performance Guide**. 2024. Disponível em: <https://about.gitlab.com/topics/devops/dora-metrics/>. Acesso em: 20 jun. 2026.

GRINSHPUN, K. **Managed Kubernetes Compared: OKE vs EKS vs GKE vs AKS**. Naviteq, 2026. Disponível em: <https://www.naviteq.io/blog/oke-vs-eks-vs-gke-vs-aks-kubernetes-compared/>. Acesso em: 20 jun. 2026.

Jenkins Project. **Jenkins on Kubernetes**. 2024. Disponível em: <https://www.jenkins.io/doc/book/installing/kubernetes/>. Acesso em: 20 jun. 2026.

Red Hat. **Deploy Helm Charts with Jenkins CI/CD in Red Hat OpenShift 4**. 2021. Disponível em: <https://developers.redhat.com/articles/2021/05/24/deploy-helm-charts-jenkins-cicd-red-hat-openshift-4>. Acesso em: 20 jun. 2026.

Spacelift. **How to Run Jenkins on Kubernetes**. 2024. Disponível em: <https://spacelift.io/blog/jenkins-kubernetes>. Acesso em: 20 jun. 2026.

SquareOps. **Best Practices for Managing Jenkins Pipelines in Kubernetes Environments**. 2024. Disponível em: <https://squareops.com/blog/best-practices-for-managing-jenkins-pipelines-in-kubernetes-environments/>. Acesso em: 20 jun. 2026.

Stackademic. **Modernizing Jenkins: From Static Agents to Kubernetes Dynamic Pods.** 2024. Disponível em: <https://blog.stackademic.com/modernizing-jenkins-from-static-agents-to-kubernetes-dynamic-pods-fbda3f897018>. Acesso em: 20 jun. 2026.

Tasrie IT Services. **EKS vs AKS vs GKE: We Migrated 50 Clusters – Here's What Won.** 2026. Disponível em: <https://tasrieit.com/blog/eks-vs-aks-vs-gke-comparison-2026>. Acesso em: 20 jun. 2026.

WASIKE, B. How to set up jenkins ci/cd on kubernetes cluster using helm. **Simple Talk**, 2023. Disponível em: <https://www.red-gate.com/simple-talk/devops/ci-cd/how-to-set-up-jenkins-ci-cd-on-kubernetes-cluster-using-helm/>. Acesso em: 20 jun. 2026.